T E A M

P R E S E N T A T I O N

1. Khethavath Sunil Naik

2. Charith

3. Ganesh

4. Karthik

5. Devendar

6. Kushagra

GRASSCUTTER ROVER

P R O J E C T

Raspberry Pi

Arduino

Intel RealSense D435i

YOLOv8

Project Overview

Project Vision

Develop an grasscutting rover capable of identifying weeds vs crops in real-time using computer vision.

Team Coordination

Managed a 6-member team across hardware, software, and ML domains — aligning timelines and integration.

System Architecture

Designed the overall pipeline: RealSense camera input → YOLOv8 detection → Raspberry Pi Decision Making → Arduino actuation.

Computer Vision: YOLOv8 Detection

Camera Input → Image Preprocessing → YOLOv8 → Classification

YOLOv8	2 Classes	Real-Time
Model Architecture	Weed vs Crop	Inference Speed

Key Technical Details

- Trained YOLOv8 on a custom agricultural dataset collected from local farms and kaggle
- Bounding-box detection differentiates weeds from crops with high confidence
- Model integrated with Intel RealSense D435i for spatial awareness
- Inference runs on Raspberry Pi, triggering rover cutting mechanism via Arduino

System Integration & Outcomes

Detection Accuracy

Achieved reliable weed vs crop classification on field test images using trained YOLOv8 model.

Full System Integration

Successfully integrated CV pipeline with Raspberry Pi, Arduino, and RealSense depth camera.

Project Milestones

Coordinated all 6 team members to deliver hardware, firmware, and AI components on schedule.

Dataset Overview

Dataset 1: Roboflow (kaggle)

Total: 1,435 images
Training: 1,260
Validation: 118
Test: 57

Dataset 2: Custom (farm visits)

Total: 1,736 images
Training: 1,245
Validation: 247
Test: 244

Combined Total

3,171 images across both datasets for comprehensive weed detection model training and evaluation.

Farm Visits

Physically visited multiple farms to photograph real-world crops and weeds under natural lighting conditions.

Image Capture

Captured hundreds of labeled images across different growth stages, angles, and weather conditions.

Dataset Labeling

Annotated every image with bounding boxes, distinguishing weed vs crop classes for supervised training.

Model Training Process

Field Images & Kaggle

Data Source

2 Classes

Weed / Crop

YOLOv8

Framework

Data Split

Divided dataset into training, validation, and test sets.

Augmentation

Applied flips, rotation, and brightness changes to improve generalization.

Training

Ran YOLOv8 training on labeled farm images; monitored loss curves.

Validation

Tested model on unseen farm images to validate field performance.

Field Data Insights & Impact

Real-World Data

Unlike synthetic datasets, field-captured photos ensured the model generalizes to actual farm environments.

Diverse Samples

Images captured across varying lighting, soil types, and crop densities for robust coverage.

Labeling Precision

Careful annotation of bounding boxes helped the model learn accurate class boundaries.

CAD Design & Mechanical Planning

Chassis Design

Designed the entire rover chassis in CAD — optimizing dimensions for field traversal and blade mounting.

Component Layout

Planned physical placement of Raspberry Pi, Arduino, battery, motors, and camera mount inside the frame.

Material Selection

Chose lightweight yet durable materials to maintain rover balance and motor efficiency.

Assembly Process

Phase 1

Frame Construction

Cut and assembled the base frame according to CAD specifications.

Phase 2

Motor Mounting

Installed drive motors at each wheel point for 2-wheel rover mobility.

Phase 3

Electronics Bay

Created a protected enclosure for electronics to prevent dust and moisture.

Phase 4

Blade Mount

Designed and 3D-printed a custom mount to securely attach the cutting blade to the motor shaft.”

Final Rover Structure

Structural Integrity

Rover withstood field tests without flexing or mechanical failures.

Compact Form Factor

Designed to navigate narrow farm rows with minimal footprint.

Integration Ready

All bays precisely sized so electronics fit cleanly without modification.

Arduino: Role in the Rover

Motor Control

Arduino nano drives the DC motors via motor driver (L298N), enabling forward, reverse, and steering.

Open loop control

Implemented open loop control on these DC motors enabling to move the base of the robot

Pi-Arduino Bridge

Receives serial commands from Raspberry Pi and translates them into motor actuation signals.

Arduino Configuration Details

Communication Flow: Raspberry Pi → Serial (USB/UART) → Arduino → Motors

PWM Motor Speed

Used analogWrite() with PWM pins to control motor speed for variable rover velocity.

Direction Control

GPIO HIGH/LOW signals routed through L298N driver for directional steering.

Serial Protocol

Defined custom command bytes (e.g. F/B/L/R/S) for Pi-to-Arduino instructions.

Safety Logic

Implemented stop-on-disconnect: rover halts if serial signal is lost.

Testing & Validation

Hardware-in-Loop Testing

Verified Arduino responses to serial commands before full rover integration.

Motor Calibration

Adjusted motors to maintain equal speed on both sides.

Communication Reliability

Stress-tested serial link under different baud rates; settled on 9600 baud for stability.

YOLOv8 Training Pipeline

Environment Setup

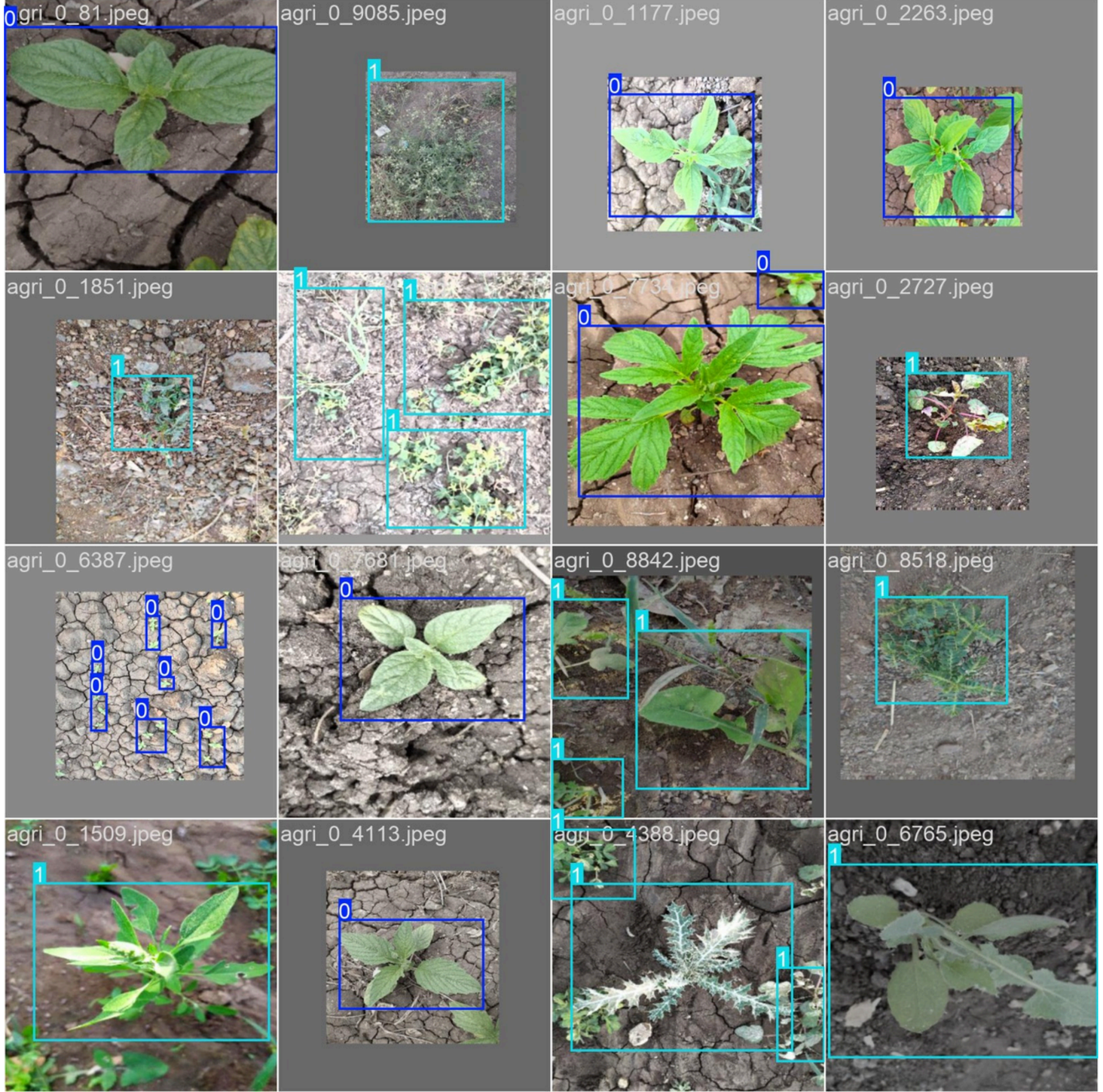
Configured Python environment with Ultralytics YOLOv8, CUDA (GPU acceleration), and required libraries.

Dataset Preparation

Organized dataset in YOLO format: images and corresponding label .txt files per class.

Training Configuration

Tuned epochs, batch size, learning rate, and image size for optimal model convergence.


agri_0_81.jpeg
agri_0_9085.jpeg
agri_0_1177.jpeg
agri_0_2263.jpeg
agri_0_1851.jpeg
agri_0_2727.jpeg
agri_0_6387.jpeg
agri_0_8842.jpeg
agri_0_8518.jpeg
agri_0_1509.jpeg
agri_0_4113.jpeg
agri_0_6765.jpeg

Hyperparameter Tuning

Parameter	Value Used	Purpose
Epochs	100	Training iterations
Batch Size	16	Samples per update
Image Size	640×640	Input resolution
Learning Rate	0.01	Optimizer step size
Optimizer	SGD	Gradient descent method

Experiments & Iterations

Multiple training runs were conducted with varying epochs and augmentations. Loss curves were monitored to detect overfitting, and the best checkpoint was selected for deployment.

Model Evaluation & Deployment

Performance Metrics

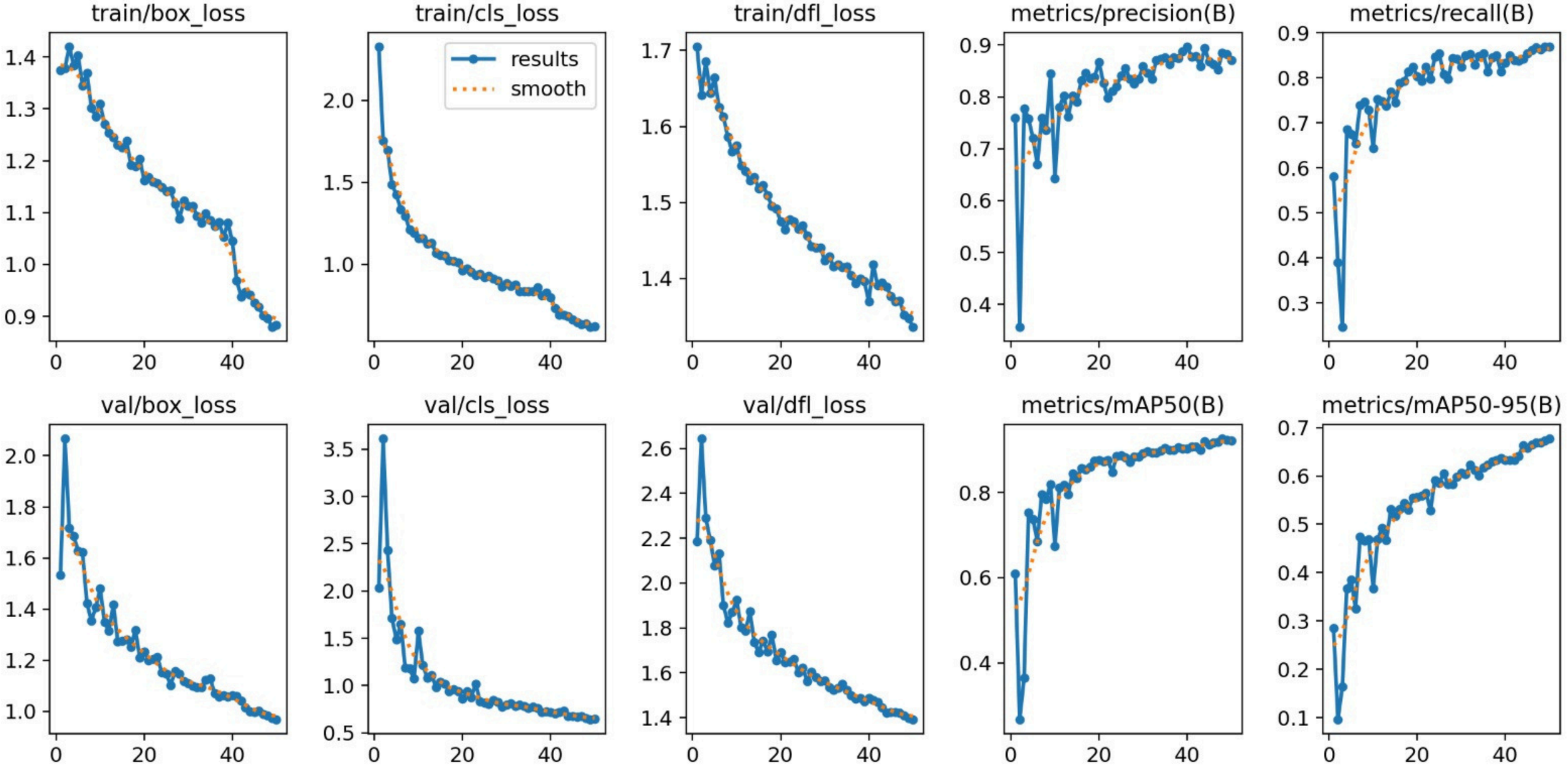
Evaluated using mAP (mean Average Precision), precision, and recall on the validation dataset.

Test Set Validation

Ran inference on held-out farm images not seen during training — confirmed generalization.

Model Export

Exported trained model to ONNX/TensorRT format for efficient deployment on Raspberry Pi.


train/box_loss
train/cls_loss
results
smooth
train/dfl_loss
metrics/precision(B)
metrics/recall(B)
val/box_loss
val/cls_loss
val/dfl_loss
metrics/mAP50(B)
metrics/mAP50-95(B)

Hardware Architecture

Raspberry Pi 4

Central computing unit — runs Python, YOLOv8 inference, and RealSense camera feed.

Intel RealSense D435i

Depth camera providing RGB + depth frames for spatial awareness of the environment.

Arduino Uno

Low-level controller for motor PWM signals, receiving commands from Raspberry Pi.

Power System

Li-Po battery with voltage regulators providing stable 5V/12V rails to all components.

Wiring & Electronics Integration

GPIO Wiring

Connected Raspberry Pi GPIO to Arduino via serial UART for command relay.

Motor Driver

Wired L298N motor driver to Arduino PWM outputs and connected to DC motors.

Camera USB

Intel RealSense connected to Raspberry Pi via USB 3.0 for high-bandwidth video.

Power Distribution

Implemented fused power rails with decoupling capacitors for stable operation.

System Testing & Final Rover

Power Stability

Verified stable voltage under load — no brownouts during rover operation or inference.

Hardware Debugging

Diagnosed and resolved motor jitter, USB connection drops, and I2C conflicts during testing.

Complete System Test

Ran full end-to-end tests: camera input → detection → serial command → motor response.

CONCLUSION

In conclusion, our Grasscutting Rover project successfully combines computer vision, machine learning, and embedded systems to support smart agriculture. Using the YOLOv8 model, Raspberry Pi, Arduino, and Intel RealSense camera, we developed a system capable of real-time weed and crop detection. This project helped us gain practical experience in robotics, hardware integration, and ML-based detection systems, while also demonstrating how technology can improve efficiency in modern farming.

THANK YOU

Grasscutting Rover — Team Presentation

Khethavath Sunil Naik

Team Lead & Computer Vision and Model Training

Charith

Dataset & Training & hardware

Ganesh

CAD & Assembly & yoloV8 model

Karthik

Arduino Configuration & raspberrypi5& dataset training

Devendar

YOLOv8 Training

Kushagra

Hardware

FAQ